



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 1-I

Seguimiento de caminos explícitos

Navegación punto a punto con control proporcional

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · CoppeliaSim · Pioneer 3-DX

Índice

1. Objetivo	3
2. Implementación	3
3. Pruebas	4
3.1. Influencia de la ganancia K	4
4. Comentarios finales	5

1. Objetivo

Completar el método `controlLoop()` del nodo `nav_p2p.cpp` (paquete `seg_tray`) para que el robot Pioneer P3DX simulado en Coppeliasim recorra una secuencia de waypoints definida en `nav_params.yaml` mediante navegación punto a punto con control proporcional de rumbo. Una vez verificado el recorrido completo, modificar la ganancia K y comentar cómo influye en la trayectoria del robot.

2. Implementación

El nodo `nav_p2p_node` ya proporcionado en `seg_tray` se encarga de cargar los waypoints del YAML, leer la odometría y publicar comandos de velocidad a 40 Hz. Lo único que hay que completar es el bloque indicado dentro de `controlLoop()` con las ecuaciones del enunciado. Las decisiones no obvias son:

Cambio a coordenadas locales del vehículo. El error de orientación ϕ_e se obtiene más directamente expresando el waypoint en el sistema de referencia del robot. Aplicando la rotación del enunciado:

$$x_{P_l} = \cos \phi_o (x_P - x) + \sin \phi_o (y_P - y), \quad y_{P_l} = -\sin \phi_o (x_P - x) + \cos \phi_o (y_P - y),$$

queda $\phi_e = \text{atan2}(y_{P_l}, x_{P_l})$, que ya está en el rango correcto sin necesidad de normalizaciones explícitas.

Control proporcional y cinemática inversa. La curvatura comandada es $\gamma = K \phi_e$ y las velocidades angulares de cada rueda se obtienen aplicando directamente las ecuaciones del enunciado:

$$\omega_i = \frac{v(1 - K \gamma)}{R}, \quad \omega_d = \frac{v(1 + K \gamma)}{R}.$$

A partir de (ω_i, ω_d) se recuperan las velocidades del vehículo (v, ω) con la cinemática directa y se publican en `cmd_vel`.

Cambio de waypoint a 1 m. El waypoint se considera alcanzado cuando la distancia euclídea al punto objetivo es inferior a 1 m:

$$\sqrt{(x_P - x)^2 + (y_P - y)^2} < 1 \text{ m},$$

tal y como indica el enunciado, y se pasa al siguiente. Cuando no quedan waypoints, se publica $v = \omega = 0$.

3. Pruebas

Tras lanzar la escena `p2p_scene.ttt` en CoppeliaSim y ejecutar `ros2 launch seg_tray nav_p2p_launch.py`, el robot recorre los 7 waypoints del fichero de configuración a $v = 1,2\text{ m s}^{-1}$ y completa el camino. La Figura 1 muestra el recorrido sobre el plano del simulador.

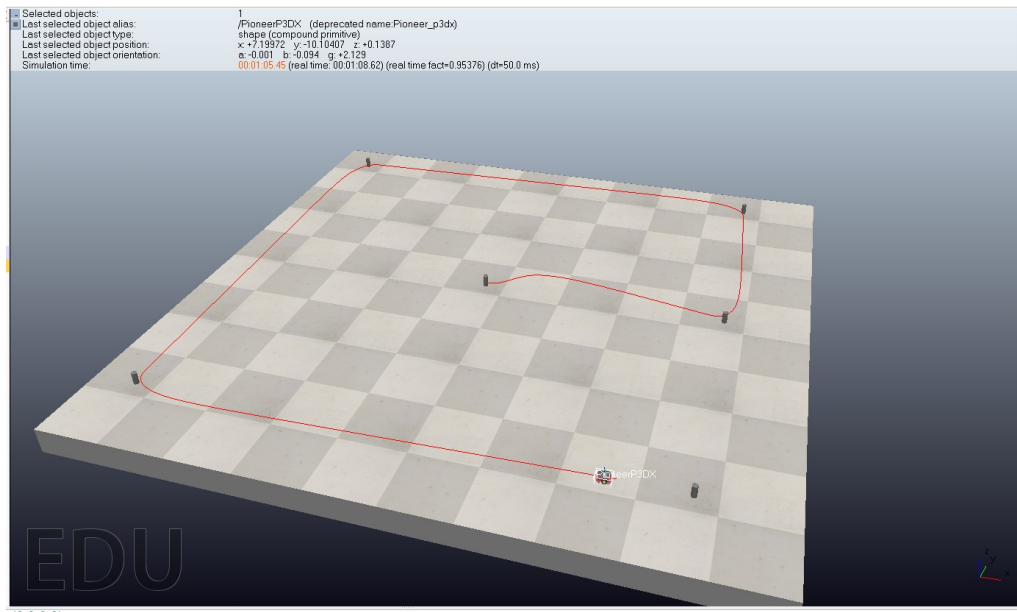


Figura 1: Recorrido rectangular del Pioneer P3DX con control P2P en Coppelia-Sim.

3.1. Influencia de la ganancia K

El enunciado pide modificar la ganancia y comentar el efecto sobre la trayectoria:

- Con K **pequeño** (e.g. $K = 1$) el robot corrige el rumbo lentamente y se aleja lateralmente del waypoint antes de orientarse, dibujando arcos amplios entre puntos consecutivos.
- Con K **moderado** (e.g. $K = 4$) la respuesta es rápida y la trayectoria entre waypoints es prácticamente recta, con curvas suaves en los cambios de waypoint.
- Con K **grande** (e.g. $K \gtrsim 6$) la curvatura comandada se vuelve excesiva y aparecen oscilaciones porque el error angular cambia de signo de un ciclo al siguiente.

El valor $K = 4$ ofrece un buen compromiso entre rapidez y estabilidad para este recorrido.

4. Comentarios finales

El controlador implementado cumple lo que pide el enunciado: el robot sigue la secuencia de waypoints con control proporcional sobre el error de orientación calculado en coordenadas locales, y la ganancia K actúa como esperado según el análisis del parámetro. La estructura del nodo proporcionado ($1,2 \text{ m s}^{-1}$, periodo 25 ms , tolerancia 1 m) es suficiente para completar el recorrido en el escenario sin obstáculos.